

Test Plan

OldPhonePad Coding Challenge

Document ID	TP-OldPhonePad-2026-01
Version	1.0
Date	2026-06-01
Author	Mauricio Batista
Project	OldPhonePad Coding Challenge — Technical Assessment
System Under Test	OldPhonePad(string input) — C# method
Test Framework	xUnit / .NET 8

1. Introduction & Scope

This Test Plan describes the testing strategy, scope, approach, and deliverables for the OldPhonePad method, developed as part of a technical coding challenge. The method simulates the input mechanism of an old mobile phone keypad, where repeated key presses cycle through the letters assigned to each key, a space character separates consecutive presses of the same key, the * key acts as backspace, and the # key sends (terminates) the input sequence.

1.1 Objectives

- Verify that all 26 defined test cases produce the correct output.
- Confirm correct handling of boundary and edge-case inputs (empty string, null, excess backspaces, wrap-around cycling).
- Validate that invalid inputs trigger the expected exceptions (ArgumentNullException).
- Ensure the method is robust, predictable, and meets the specification in all scenarios.

1.2 Out of Scope

- Performance or load testing.
- UI or integration testing — the method is tested in isolation as a pure unit.
- Any module other than OldPhonePad(string input).

2. Test Items

The following item constitutes the configuration under test:

Item	Identifier	Version	Language / Runtime
OldPhonePad method	OldPhonePad(string input)	.NET 8	C# / xUnit

3. Features to Test

The following functional areas of the OldPhonePad method are covered by the test suite:

ID	Feature	Description
F-01	Single key press	One press of a key returns the first letter assigned to it (e.g. 2# → A).
F-02	Multi-press cycling	Repeated presses cycle through the key's letters; pressing beyond the last letter wraps back to the first (e.g. 7777# → S, 77777# → P).
F-03	Space separator	A space between presses of the same key commits the current letter and starts a new one (e.g. 2 2# → AA).
F-04	Key 0 as space	Key 0 inserts a literal space character in the output (e.g. 2022# → A B).
F-05	Key 1 special characters	Key 1 cycles through special characters: & ' (e.g. 1# → &, 11# → ', 111# →).
F-06	Backspace (*)	The * character removes the last committed character from the buffer. Multiple consecutive * chars remove multiple characters. Backspace on an empty buffer is safely ignored.
F-07	Send / termination (#)	The # character ends input and returns the accumulated string result.
F-08	Null / empty input	Null input raises ArgumentNullException. Empty string and strings containing only # or spaces return an empty string.

4. Approach / Strategy

4.1 Test Strategy

Testing is performed exclusively at the unit level using the Black-Box technique: test cases are derived from the method's specification without knowledge of its internal implementation. Each test case provides a specific input string and asserts the expected output or exception.

4.2 Test Categories

Category	Test Count	Coverage Goal
PDF / Spec Examples	4	Validate the exact examples provided in the challenge specification.
Single Key Press	3	Confirm first-letter selection for keys 2, 3, and 9.

Category	Test Count	Coverage Goal
Space Separator	2	Verify same-key separation and key-0 space insertion.
Cycling Wrap-Around	3	Confirm wrap-around behaviour at the end of each key's letter set.
Backspace	6	Cover single, multiple, chained, and mid-sequence backspace cases.
Backspace Until Empty	1	Confirm safe handling when backspaces exceed typed characters.
Key 1 Special Chars	3	Verify all three special characters assigned to key 1.
All Keys	1	Smoke test covering one press of every key in sequence.
Defensive / Edge Cases	4	Null input, empty string, only #, and only spaces + #.

4.3 Defect Severity Levels

Defects found during execution are classified using the following severity scale:

Level	Severity	Definition
1	Critical	Method crashes, throws an unhandled exception, or produces completely wrong output. Testing is suspended until resolved.
2	High	A core feature produces incorrect results for a significant subset of inputs. Testing may continue on unaffected areas.
3	Medium	A feature behaves incorrectly in a specific edge case but does not block overall execution.
4	Low	Minor inconsistency or cosmetic issue with negligible impact on correctness.

4.4 Exit Criteria

- All 26 test cases have been executed.
- No open defects with severity Level 1 or Level 2.
- All Level 3+ defects are documented, triaged, and have an assigned resolution.
- The Test Execution Report and Defect Log are complete and reviewed.

5. Deliverables

Document	Description	Status
Test Plan (this document)	Scope, strategy, approach, and environment details.	Complete
Test Case Design Document	26 test cases with inputs, expected outputs, preconditions, and test types.	Complete

Document	Description	Status
Test Execution Report	Execution results for all 26 test cases, including pass/fail status and actual output.	Complete

6. Environment

All tests are executed in a local development environment with the following configuration:

Operating System	Windows / Linux / macOS (cross-platform)
Runtime	.NET 8
Language	C# 12
Test Framework	xUnit 2.x
Test Runner	dotnet test CLI
IDE	VS Code
Version Control	Git
Execution Time	~2.7 seconds (26 test cases)
External Dependencies	None — OldPhonePad is a pure in-memory method with no I/O or DB calls

6.1 Notes

- No dedicated test environment is required; the method has no external dependencies.
- Tests can be reproduced on any machine with .NET 8 SDK installed by running: dotnet test
- Test results are deterministic — the same input always produces the same output.